

Docker Compose: The Complete Guide (Theory → Practice)

A step-by-step journey from "what is this?" to production-level usage, with heavily commented examples, common mistakes, and interview Q&A.

PART 0: THE THEORY — What Problem Does Docker Compose Solve?

The problem

When you use plain Docker, running **one** container is easy:

```
bash
docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres
```

But real applications are rarely one container. A typical web app might need:

- A backend API container
- A frontend container
- A database container
- A cache (Redis) container
- Maybe a message queue

Running each of these manually with `docker run` means:

- Typing long commands with many flags, every single time
- Manually creating a Docker network so containers can talk to each other
- Manually starting things in the right order
- No single place to see "this is my whole application"

The solution: Docker Compose

Docker Compose is a tool that lets you define your **entire multi-container application** in a single YAML file (`compose.yaml`, also historically called `docker-compose.yml`). You describe:

- What containers (services) you need
- How they're built or which image to use
- How they connect to each other (networking)
- What data they need to persist (volumes)
- What environment variables/config they need

Then you bring the whole thing up or down with **one command**.

Key mental model

Think of Compose as a **blueprint + remote control**:

- The YAML file = blueprint (declarative description of desired state)
- The `docker compose` CLI = remote control (turns that blueprint into running containers)

Important terminology

Term	Meaning
Service	One container definition in Compose (e.g., "web", "db"). Not the running container itself — the <i>spec</i> for it.
Project	The whole application defined by a compose file (named after the folder by default, or via <code>-p</code>).
Image	The read-only template a container is built from.
Container	A running (or stopped) instance of an image.
Volume	Persistent storage that survives container restarts/removal.
Network	Virtual network Compose creates so services can find each other by name.

Docker Compose V1 vs V2 (know this!)

- **V1**: separate Python tool, invoked as `docker-compose` (with a hyphen). Deprecated.
- **V2**: rewritten in Go, built into the Docker CLI, invoked as `docker compose` (a space, as a subcommand). This is the modern standard — **use this**.

Also note: newer Compose files no longer need a `version:` key at the top (it's deprecated/ignored by the modern spec). If you see `version: "3.8"` in older tutorials, it's harmless but unnecessary today.

STEP 1 (Beginner): Installation & Your First Compose File

1.1 Check you have it

Docker Compose V2 ships with Docker Desktop and modern Docker Engine installs.

```
bash
```

```
docker compose version
```

```
# Should print something like: Docker Compose version v2.x.x
```

If this fails, install/update Docker Desktop (Mac/Windows) or the `docker-compose-plugin` package (Linux).

1.2 Your first `compose.yaml`

Let's containerize a single Nginx web server — the "hello world" of Compose.

Create a folder and file:

```
bash
```

```
mkdir my-first-compose && cd my-first-compose
```

```
touch compose.yaml
```

`compose.yaml`:

```
yaml
```

```
# The top-level key "services" lists every container we want to run.
```

```
services:
```

```
    # "web" is the NAME we are giving this service.
```

```
    # This name also becomes its hostname on the internal Docker network.
```

```
    web:
```

```
        image: nginx:latest    # Use the official Nginx image from Docker Hub
```

```
        ports:
```

```
            - "8080:80"
```

```
                # Map "host_port:container_port"
```

```
                # i.e., http://localhost:8080 -> port 80 inside the
```

1.3 Bring it up

```
bash
```

```
docker compose up
```

What happens:

1. Compose reads `compose.yaml` in the current directory
2. It pulls the `nginx:latest` image if you don't have it locally
3. It creates a container named something like `my-first-compose-web-1`
4. It creates a default network for the project
5. Logs stream directly to your terminal (this blocks your terminal — Ctrl+C stops it)

Visit `http://localhost:8080` — you'll see the Nginx welcome page.

1.4 Run it in the background (detached mode)

```
bash

docker compose up -d
# -d = "detached": runs containers in the background, gives you your terminal b
```

1.5 See what's running

```
bash

docker compose ps
# Shows containers belonging to THIS compose project (not all Docker containers
```

1.6 Stop and remove everything

```
bash

docker compose down
# Stops containers AND removes containers + the default network it created
# (Does NOT remove volumes by default — that's a safety feature)
```

Checkpoint — you now understand: a compose file describes services, `up` creates/starts them, `down` tears them down.

STEP 2 (Beginner-Intermediate): The Core Command Toolkit

These are the commands you'll use daily. Practice each one against the Nginx example above.

bash

Start everything in the background

`docker compose up -d`

View logs from ALL services, following in real time (like `tail -f`)

`docker compose logs -f`

View logs from just ONE service

`docker compose logs -f web`

List running containers for this project (plus their status/ports)

`docker compose ps`

Stop containers WITHOUT removing them (keeps them, just paused/exited)

`docker compose stop`

Start them again (containers already exist, just restart them)

`docker compose start`

Restart a service (useful after changing an env var or config file)

`docker compose restart web`

Run a command INSIDE a running container (like SSH-ing in)

`docker compose exec web bash`

-> drops you into an interactive shell inside the "web" container

Use "sh" instead of "bash" if the image is Alpine-based (no bash by default)

Rebuild images (needed after you change a Dockerfile the service uses)

`docker compose build`

Build AND start in one go

`docker compose up -d --build`

Tear everything down: stop + remove containers + remove default network

`docker compose down`

Tear down AND remove named volumes too (DESTROYS persisted data – be careful!)

`docker compose down -v`

Validate your compose file syntax without running anything

`docker compose config`

Rule of thumb: `up`/`down` manage the whole lifecycle; `start`/`stop` just pause/resume existing containers without destroying them.

STEP 3 (Intermediate): Multi-Service Apps & Networking

This is where Compose starts to shine. Let's build a realistic setup: a Python Flask API + a Postgres database.

3.1 Project structure

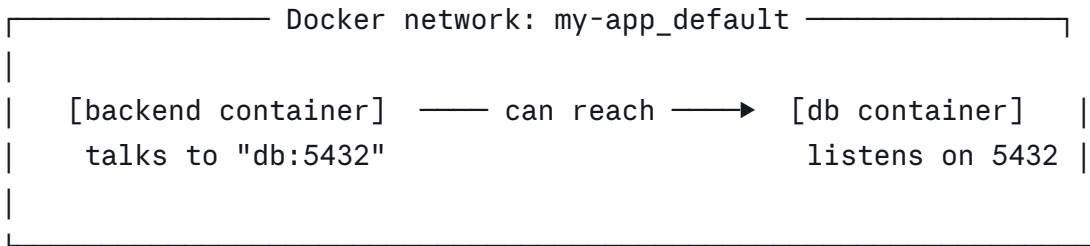
```
my-app/  
├─ compose.yaml  
├─ backend/  
│   └─ Dockerfile  
│   └─ app.py  
│   └─ requirements.txt
```

3.2 The compose file

```
yaml  
  
services:  
  
  backend:  
    build: ./backend          # Build from the Dockerfile in ./backend  
                                # (instead of pulling a pre-made image)  
    ports:  
      - "5000:5000"  
    environment:  
      # These become environment variables INSIDE the backend container.  
      # Notice "db" as the hostname – that's the SERVICE NAME, not "localhost"!  
      DATABASE_URL: postgresql://myuser:mypassword@db:5432/mydb  
    depends_on:  
      - db                    # Ensures "db" is STARTED before "backend" starts  
                                # (NOT the same as "db is ready" – see Step 6 for  
  
  db:  
    image: postgres:16  
    environment:  
      POSTGRES_USER: myuser  
      POSTGRES_PASSWORD: mypassword  
      POSTGRES_DB: mydb  
    # We deliberately do NOT expose ports to the host here –  
    # the backend can reach it internally without us opening it to our machine.
```

3.3 The crucial networking concept

By default, Compose creates **one network per project**, and every service on it can reach every other service **by service name**, like a hostname. This is why the backend connects to `db:5432` and NOT `localhost:5432` or `127.0.0.1:5432`.



This internal DNS resolution is handled automatically by Docker — you never configure it manually.

3.4 Building and running

```
bash

docker compose up -d --build
docker compose logs -f backend    # watch the backend's logs specifically
```

Common beginner trap: trying to connect to the database using `localhost` from inside the backend container. `localhost` INSIDE a container refers to that container itself, not your host machine or other containers. Always use the service name.

STEP 4 (Intermediate): Environment Variables & `.env` Files

Hardcoding secrets/config directly in `compose.yaml` is bad practice. Let's externalize them.

4.1 Create a `.env` file (same directory as `compose.yaml`)

```
env

# .env
POSTGRES_USER=myuser
POSTGRES_PASSWORD=supersecret123
POSTGRES_DB=mydb
BACKEND_PORT=5000
```

Compose **automatically** loads a file named exactly `.env` in the project directory — no extra config needed.

4.2 Reference those variables in `compose.yaml`

yaml

```
services:
  backend:
    build: ./backend
    ports:
      - "${BACKEND_PORT}:5000"    # ${VAR} pulls from .env (or your shell enviro
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
```

4.3 Passing an `env_file` to a container (different from the project `.env`!)

There's an important distinction:

- The project-root `.env` file → used for **variable substitution** inside `compose.yaml` itself (the `${VAR}` syntax).
- `env_file:` → injects variables **directly into the container's environment**, without touching the compose file's own variables.

yaml

```
services:
  backend:
    build: ./backend
    env_file:
      - ./backend/.env.production    # loads ALL these as env vars inside the co
```

4.4 Always `.gitignore` your secrets

gitignore

```
.env
*.env.production
```

Commit a `.env.example` with dummy values instead, so teammates know what's needed.

STEP 5 (Intermediate-Advanced): Volumes — Persisting & Sharing Data

Containers are ephemeral — delete the container, lose the data, unless you use **volumes**.

5.1 Named volumes (managed by Docker — best for databases)

```
yaml

services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      # "pgdata" (named volume) : "/var/lib/postgresql/data" (path INSIDE container)
      - pgdata:/var/lib/postgresql/data

# Named volumes must be declared at the top level too:
volumes:
  pgdata:      # Docker manages the actual storage location on your machine
```

Now, even if you `docker compose down` and `up` again, your database data survives (as long as you don't add `-v`).

5.2 Bind mounts (best for local development — live code reload)

```
yaml

services:
  backend:
    build: ./backend
    volumes:
      # HOST_PATH : CONTAINER_PATH
      # Maps your actual local folder into the container.
      # Edit code on your machine -> changes reflect INSIDE the container instantly
      - ./backend:/app
    ports:
      - "5000:5000"
```

Named volume vs bind mount — the core difference:

	Named Volume	Bind Mount
Managed by	Docker	You (it's a real folder on your machine)
Best for	Databases, persistent app data	Local development (live code editing)
Portable	Yes	No (depends on host filesystem path)
Visible in <code>docker volume ls</code>	Yes	No

5.3 Anonymous volumes (rarely used deliberately, but good to recognize)

```
yaml
volumes:
  - /app/node_modules    # no host path, no name = anonymous volume
```

A common pattern: combine a bind mount for source code with an anonymous volume to **prevent** your host's `node_modules` (which may be empty or wrong OS build) from overwriting the container's own `node_modules`:

```
yaml
services:
  frontend:
    build: ./frontend
    volumes:
      - ./frontend:/app          # bind mount: your source code
      - /app/node_modules        # anonymous volume: protects container's own n
```

STEP 6 (Advanced): Healthchecks, Restart Policies & Startup Order

6.1 The problem with plain `depends_on`

`depends_on: [db]` only waits for the `db` **container to start**, not for Postgres **inside it to be ready to accept connections**. This causes a classic bug: your backend crashes on startup because it tried to connect too early.

6.2 The fix: healthchecks + `condition`

yaml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    healthcheck:
      # Command Docker runs INSIDE the container to check health
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s      # check every 5 seconds
      timeout: 3s       # fail if the check takes longer than 3s
      retries: 5         # mark "unhealthy" after 5 consecutive failures
      start_period: 10s # grace period before failures start counting (startup)

  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy # wait until db passes its healthcheck, no
```

6.3 Restart policies (resilience)

yaml

```
services:
  backend:
    build: ./backend
    restart: unless-stopped
    # Options:
    # "no"          -> never restart (default)
    # "always"      -> always restart, even after manual stop, even after reb
    # "on-failure"   -> only restart if it exits with a non-zero (error) code
    # "unless-stopped" -> like "always", but respects a manual "docker compose s
```

STEP 7 (Advanced): Multiple Compose Files, Profiles & Overrides

7.1 Override files (dev vs prod configs from one base)

Compose automatically merges `compose.yaml` with `compose.override.yaml` if present.

`compose.yaml` (base — shared by everyone):

yaml

```
services:
  backend:
    build: ./backend
    environment:
      NODE_ENV: production
```

`compose.override.yaml` (auto-loaded locally, e.g., for dev — NOT committed or gitignored per team preference):

yaml

```
services:
  backend:
    environment:
      NODE_ENV: development
    volumes:
      - ./backend:/app    # live reload, only in dev
    ports:
      - "9229:9229"      # expose a debugger port, only in dev
```

Running plain `docker compose up` automatically merges both files. To explicitly target a production file instead:

bash

```
docker compose -f compose.yaml -f compose.prod.yaml up -d
# -f flags are applied in order; later files override/merge with earlier ones
```

7.2 Profiles (optional services)

Useful when some services (e.g., a debugging tool, or a seed-data job) shouldn't always start.

yaml

```
services:
  backend:
    build: ./backend

  pgadmin:
    image: dpage/pgadmin4
    profiles: ["debug"]    # only starts if the "debug" profile is explicitly ac
```

```
bash
```

```
docker compose up -d          # pgadmin is SKIPPED
docker compose --profile debug up -d  # pgadmin is INCLUDED
```

7.3 Scaling a service

```
bash
```

```
docker compose up -d --scale backend=3
# Runs 3 instances of the "backend" service (useful for load-testing locally)
# Note: you can't use a fixed host port mapping like "5000:5000" with scale > 1
# since multiple containers can't all bind to the same host port.
```

STEP 8 (Advanced/Production-Adjacent): Real-World Considerations

8.1 Secrets (better than plain environment variables for sensitive data)

```
yaml
```

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password  # Postgres image supports
secrets:
  - db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt  # a plain text file containing just the password
```

This avoids the password showing up in `docker inspect` output or process listings the way a plain env var can.

8.2 Custom networks for isolation

yaml

```
services:
  backend:
    networks: [frontend_net, backend_net]
  db:
    networks: [backend_net]      # db is ONLY reachable from backend, not from f

networks:
  frontend_net:
  backend_net:
```

8.3 Resource limits (prevent one container from starving the host)

yaml

```
services:
  backend:
    deploy:
      resources:
        limits:
          cpus: "0.50"      # max 50% of one CPU core
          memory: 512M
        reservations:
          memory: 256M      # guaranteed minimum
```

Note: the `deploy` key's resource limits are fully respected under plain `docker compose up` since Compose V2; historically `deploy` was Swarm-only.

8.4 Using Compose in CI/CD

bash

```
# Typical CI pipeline step: spin up dependencies, run tests, tear down
docker compose -f compose.test.yaml up -d --build
docker compose -f compose.test.yaml exec -T backend pytest      # -T disables pseu
docker compose -f compose.test.yaml down -v
```

COMMON BEGINNER MISTAKES

1. **Using `localhost` between containers.** Inside a container, `localhost` means *that container*. Use the service name instead (e.g., `db`), not `localhost`).
 2. **Forgetting `depends_on` doesn't mean "ready."** It only waits for the container to *start*, not for the app inside to be ready. Use healthchecks for real readiness.
 3. **Losing data because they forgot to declare a named volume.** Without a volume mapping, all data inside the container vanishes when the container is removed.
 4. **Running `docker compose down -v` carelessly.** The `-v` flag deletes volumes — including your database data. Only use it when you actually want a clean slate.
 5. **Committing `.env` files with real secrets to Git.** Always `.gitignore` them.
 6. **Editing a Dockerfile but forgetting to rebuild.** `docker compose up` alone won't rebuild an image after Dockerfile changes — you need `--build` or `docker compose build`.
 7. **Port confusion (`HOST:CONTAINER`).** `"8080:80"` means "host port 8080 maps to container port 80" — mixing up the order is a very common bug.
 8. **Not realizing bind mounts can hide files.** If you bind-mount `./backend:/app` but the image also installed dependencies into `/app/node_modules`, the empty host folder can "shadow" and wipe out that install — hence the anonymous volume trick in Step 5.3.
 9. **Using `latest` tag everywhere in "production" configs.** Makes builds non-reproducible; pin versions like `postgres:16.2` instead.
 10. **Assuming `docker-compose` (hyphenated, V1) and `docker compose` (V2) behave identically.** Small syntax and behavior differences exist; stick to V2 going forward.
-

INTERVIEW QUESTIONS (Intermediate → Advanced)

Q1: What's the difference between `docker compose up`, `start`, and `restart`?

`up` creates (if needed) and starts containers, networks, and volumes based on the compose file — it's the full lifecycle command. `start` only starts containers that **already exist** (created previously); it won't create anything new. `restart` stops and starts an existing container, useful for picking up new environment variable values or restarting a hung process, without recreating it.

Q2: How do containers in the same Compose project discover each other?

Compose creates a default bridge network per project, and Docker's embedded DNS server resolves each service's name to its container's internal IP address automatically. So a service just needs to connect to `http://servicename:port` — no manual IP management or `/etc/hosts` editing required.

Q3: Why might `depends_on` alone cause a race condition, and how do you fix it?

`depends_on` guarantees start **order**, not application **readiness** — a database container can report "started" before Postgres has finished initializing and is actually accepting connections. The fix is to add a `healthcheck` to the dependency and use `depends_on: { service: { condition: service_healthy } }` so dependents wait for a real readiness signal, or build retry/backoff logic into the dependent application itself.

Q4: Explain the difference between a named volume and a bind mount, and when you'd use each.

A named volume is storage fully managed by Docker (location abstracted away), ideal for persistent application data like databases because it's portable and decoupled from the host's filesystem layout. A bind mount maps a specific path on the host directly into the container, ideal for local development because it enables live code editing/hot-reload — but it's less portable since it depends on the host's directory structure.

Q5: How does Compose handle multiple `-f` files, and why would you use that?

Compose merges multiple files in the order given via `-f base.yaml -f override.yaml`, with later files' keys overriding/merging into earlier ones. This lets teams keep a shared base configuration and layer environment-specific overrides (dev vs. staging vs. production) without duplicating the whole file.

Q6: What happens to volumes when you run `docker compose down` vs `docker compose down -v`?

Plain `down` stops and removes containers and the default network, but **preserves** named volumes, so persistent data (like a database) survives. Adding `-v` additionally removes any volumes defined in the compose file (and anonymous volumes), permanently deleting that data — this is a common source of "I lost my database" incidents.

Q7: How would you isolate a database so only the backend can reach it, not the frontend?

Define separate custom networks (e.g., `frontend_net` and `backend_net`), attach the frontend service only to `frontend_net`, attach the backend to both, and attach the database only to `backend_net`. Since Compose's default network exposes every service to every other service, custom networks are necessary to enforce this kind of internal network segmentation.

Q8: What's the difference between the project-level `.env` file and a service's `env_file` directive?

The root `.env` file is used by Compose itself for variable **substitution** within the compose YAML (the `${VARIABLE}` syntax) — it doesn't automatically become environment variables inside a container unless referenced. `env_file:` on a specific service, in contrast, loads all key-value pairs from that file directly as environment variables **inside that container's runtime environment**, independent of variable substitution in the YAML.

Q9: Why is pinning image versions (e.g., `postgres:16.2`) instead of `postgres:latest`) important in production Compose files?

Using `latest` makes builds non-reproducible — a fresh `docker compose pull` at a different point in time could silently pull a different (possibly breaking) version, causing "works on my machine" drift between environments and making rollbacks/debugging much harder. Pinning specific versions ensures identical, reproducible environments across dev, staging, and production.

Q10: How do Compose "profiles" differ from just commenting services out of the file?

Profiles let you keep optional services (debugging tools, seed jobs, admin UIs) defined in the same file but excluded from the default `docker compose up` run unless explicitly activated with `--profile <name>`. This avoids maintaining separate files or manually commenting/uncommenting YAML, keeping one source of truth while still supporting conditional inclusion.

Q11: What's a potential pitfall of using `deploy.resources.limits` under plain (non-Swarm) `docker compose up`?

Historically, the `deploy` key was documented as Swarm-mode-only and ignored by plain `docker compose up`; while modern Compose V2 does respect `deploy.resources.limits` outside Swarm, engineers relying on older documentation or mixed tooling versions can be caught off guard if limits silently don't apply, leading to unexpected resource contention in what they assumed was a constrained environment. Always verify behavior against the Compose version actually in use.

Q12: How would you debug a container that keeps restarting in a `restart: on-failure` loop?

Check logs first with `docker compose logs <service>` (add `--tail` to limit output) to see the crash reason; if it exits too fast to inspect interactively, temporarily override the entrypoint/command (e.g., `docker compose run --entrypoint sh <service>`) to get a shell without the normal startup command running, or set `restart: "no"` temporarily so the container stays in its exited state for `docker inspect`/log analysis instead of continuously cycling.

Quick Reference Card (save this)

bash

```
docker compose up -d           # start everything in background
docker compose up -d --build   # rebuild images then start
docker compose down           # stop & remove containers + network
docker compose down -v        # ...also delete volumes (data loss!)
docker compose ps             # list this project's containers
docker compose logs -f <service> # follow logs for one service
docker compose exec <service> sh # shell into a running container
docker compose restart <service> # restart one service
docker compose build           # rebuild images without starting
docker compose config          # validate/print the resolved config
docker compose --profile X up -d # start including optional profile X
docker compose -f a.yaml -f b.yaml up -d # merge multiple compose files
```